Lab 10 programs

Name: J Sravani
Reg no: 192372141

Experiment 1: Data Exploration and Cleaning with pandas

Aim

To load, inspect, clean and summarize student marks data and visualize relationships between study hours and exam marks.

Algorithm

1. Import pandas and matplotlib.
2. Create/load student_marks.csv with student details and marks.
3. Inspect rows, shape, data types, descriptive statistics and missing values.
4. Fill numeric missing values with medians and categorical missing values with modes.
5. Remove duplicate rows and treat attendance outside 0–100 as invalid.
6. Calculate final-exam summary statistics, department means and correlation.
7. Draw histogram, department box plot and study-hours scatter plot

Program

```
import pandas as pd
# Create a sample dataset

data = {

   "Name": ["Ravi", "Sita", "Arun", "Priya", "Kiran"],

   "Age": [20, 21, None, 22, 20],

   "Marks": [85, 90, 78, None, 88]

}

df = pd.DataFrame(data)

print("Original Dataset:")

print(df)

print("\nFirst 5 Rows:")

print(df.head())

print("\nDataset Information:")

df.info()

print("\nMissing Values:")
```

```
print(df.isnull().sum())

# Fill missing numerical values with column means

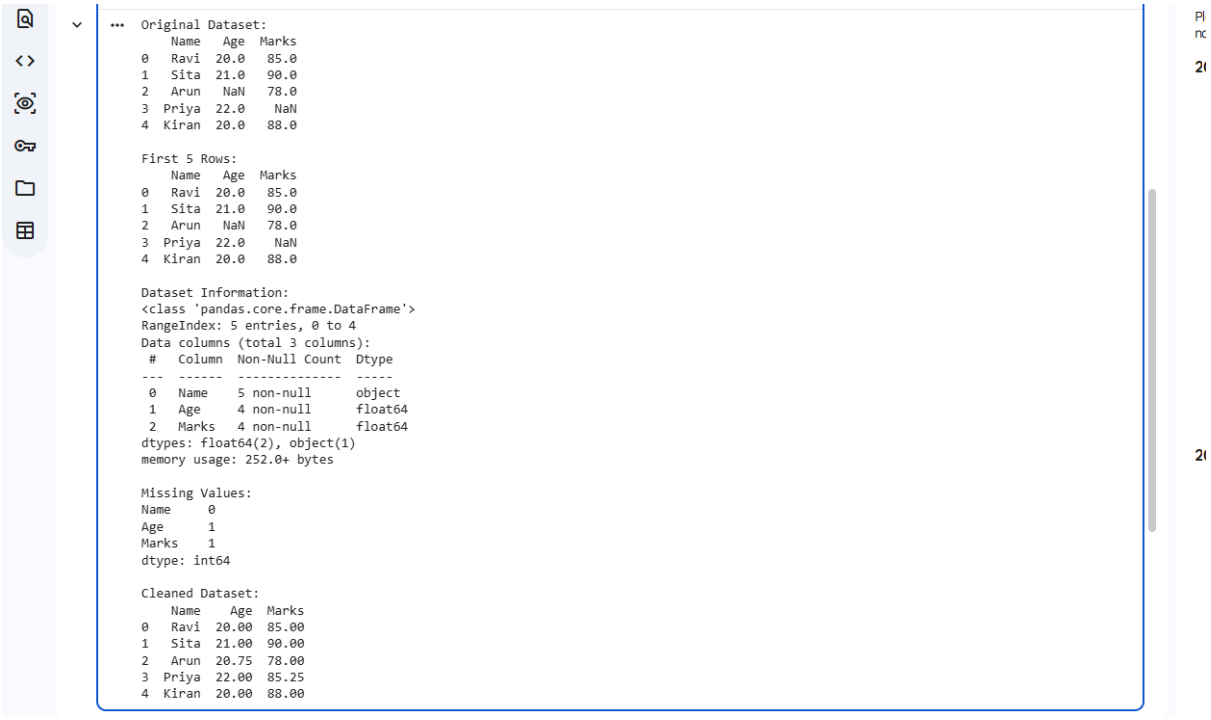
df["Age"] = df["Age"].fillna(df["Age"].mean())

df["Marks"] = df["Marks"].fillna(df["Marks"].mean())

print("\nCleaned Dataset:")

print(df)
```

OUTPUT



```
Original Dataset:
    Name   Age  Marks
0   Ravi  20.0   85.0
1   Sita  21.0   90.0
2   Arun   NaN   78.0
3  Priya  22.0    NaN
4  Kiran  20.0   88.0

First 5 Rows:
    Name   Age  Marks
0   Ravi  20.0   85.0
1   Sita  21.0   90.0
2   Arun   NaN   78.0
3  Priya  22.0    NaN
4  Kiran  20.0   88.0

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 3 columns):
 #   Column  Non-Null Count  Dtype
---  ------  --------------  -----
 0   Name    5 non-null      object
 1   Age     4 non-null      float64
 2   Marks   4 non-null      float64
dtypes: float64(2), object(1)
memory usage: 252.0+ bytes

Missing Values:
Name     0
Age      1
Marks    1
dtype: int64

Cleaned Dataset:
    Name    Age  Marks
0   Ravi  20.00  85.00
1   Sita  21.00  90.00
2   Arun  20.75  78.00
3  Priya  22.00  85.25
4  Kiran  20.00  88.00
```

Experiment 2: Semantic Network and Inheritance Queries

Aim

To represent knowledge as a semantic network and answer questions using inheritance, exception overriding and path search.

Algorithm

8. Import networkx and matplotlib.
9. Create a directed graph with animal classes and an individual Tweety.
10. Add is-a and instance-of relationships and properties such as can-fly and has-wings.
11. Implement property lookup by searching the node first and then its parents.
12. Use a local Penguin exception to override the general Bird flying property.
13. Answer at least five queries and find a path from Tweety to Animal.Draw and display the labelled graph.

Program

```
!pip -q install networkx matplotlib

import networkx as nx

import matplotlib.pyplot as plt


G = nx.DiGraph()


G.add_edge("Bird", "Animal", relation="is-a")

G.add_edge("Fish", "Animal", relation="is-a")

G.add_edge("Canary", "Bird", relation="is-a")

G.add_edge("Penguin", "Bird", relation="is-a")

G.add_edge("Tweety", "Canary", relation="instance-of")


properties = {

    "Animal": {"alive": True},

    "Bird": {"can_fly": True, "has_wings": True},

    "Fish": {"can_swim": True},

    "Canary": {"colour": "yellow"},

    "Penguin": {"can_fly": False, "can_swim": True},

    "Tweety": {}

}


def inherits(node, relation):

    queue = [node]

    visited = set()


    while queue:

        current = queue.pop(0)

        if current in visited:

            continue

        visited.add(current)


        if relation in properties.get(current, {}):

            return properties[current][relation]
```

```
        queue.extend(G.successors(current))

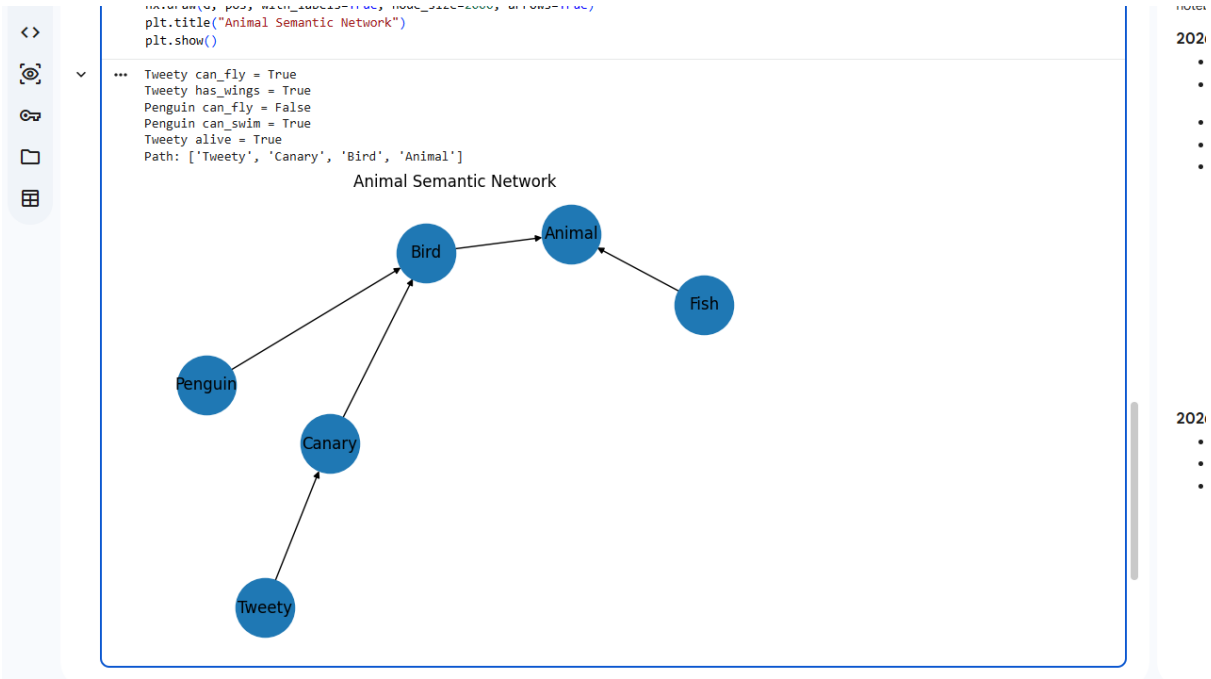
    return None

for animal, prop in [
    ("Tweety", "can_fly"),
    ("Tweety", "has_wings"),
    ("Penguin", "can_fly"),
    ("Penguin", "can_swim"),
    ("Tweety", "alive")
]:
    print(animal, prop, "=", inherits(animal, prop))

print("Path:", nx.shortest_path(G, "Tweety", "Animal"))

pos = nx.spring_layout(G, seed=7)
nx.draw(G, pos, with_labels=True, node_size=2000, arrows=True)
plt.title("Animal Semantic Network")
plt.show()
```

Output



Experiment 3: Frames with Defaults, Inheritance and If-Needed Procedures

Aim

To implement frames in Python with slots, default values, inheritance, computed values and an if-added procedure.

Algorithm

14. Create a Frame class with a name, optional parent and slots dictionary.
15. Store each slot's value, default, if-needed function and optional if-added function.
16. Implement get() to check a local value, computed procedure, default, and then parent.
17. Implement set() to store a value and execute the if-added procedure.
18. Create Vehicle, Car and MyCar frames with wheels, fuel, odometer and mileage slots.
19. Print each slot value and demonstrate a warning for negative mileage.

Program

```
class Frame:

    def __init__(self, name, parent=None):

        self.name = name

        self.parent = parent

        self.slots = {}



    def add_slot(self, slot, value=None, default=None):

        self.slots[slot] = {

            "value": value,

            "default": default

        }



    def get(self, slot):

        if slot in self.slots:

            data = self.slots[slot]



            if data["value"] is not None:

                return data["value"]



            if data["default"] is not None:

                return data["default"]
```

```
        if self.parent:
            return self.parent.get(slot)

        return None

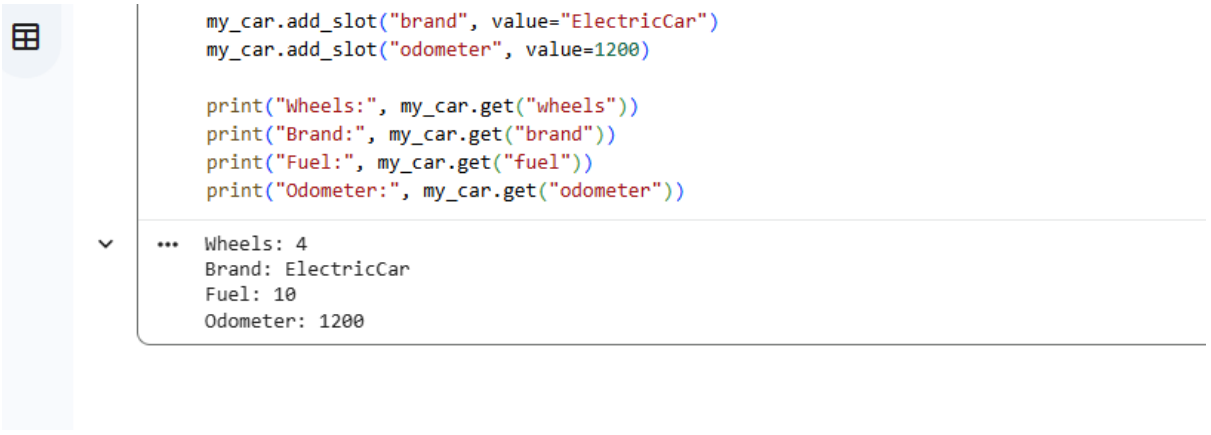
vehicle = Frame("Vehicle")
vehicle.add_slot("wheels", default=4)
vehicle.add_slot("fuel", default=10)

car = Frame("Car", vehicle)
car.add_slot("brand", default="Generic")

my_car = Frame("MyCar", car)
my_car.add_slot("brand", value="ElectricCar")
my_car.add_slot("odometer", value=1200)

print("Wheels:", my_car.get("wheels"))
print("Brand:", my_car.get("brand"))
print("Fuel:", my_car.get("fuel"))
print("Odometer:", my_car.get("odometer"))
```

Output



Experiment 4: Rule-Based System with Forward Chaining

Aim

To build a production system that repeatedly applies IF–THEN rules to known facts using forward chaining.

Algorithm

1. **Represent each rule as a set of conditions and one conclusion.**
2. **Store initial facts in working memory.**
3. **Find rules whose conditions are all present and whose conclusions are new.**
4. **Choose a matching rule, add its conclusion, and repeat until no rule can fire.**
5. **Print the conflict set, selected rule and current facts in each cycle.**
6. **Test the system with three different initial fact sets.**

Program

```
rules = [
    ({"income_ok", "credit_good"}, "loan_eligible"),
    ({"loan_eligible", "documents_ok"}, "loan_approved"),
    ({"income_low"}, "needs_guarantor"),
    ({"needs_guarantor", "guarantor_available"}, "loan_eligible"),
    ({"loan_approved"}, "notify_customer")
]


def forward_chain(initial_facts):
```

```
    facts = set(initial_facts)

    while True:
        fired = False

        for conditions, conclusion in rules:
            if conditions.issubset(facts) and conclusion not in facts:
                print("Rule fired:", conditions, "->", conclusion)
                facts.add(conclusion)
                fired = True
                break

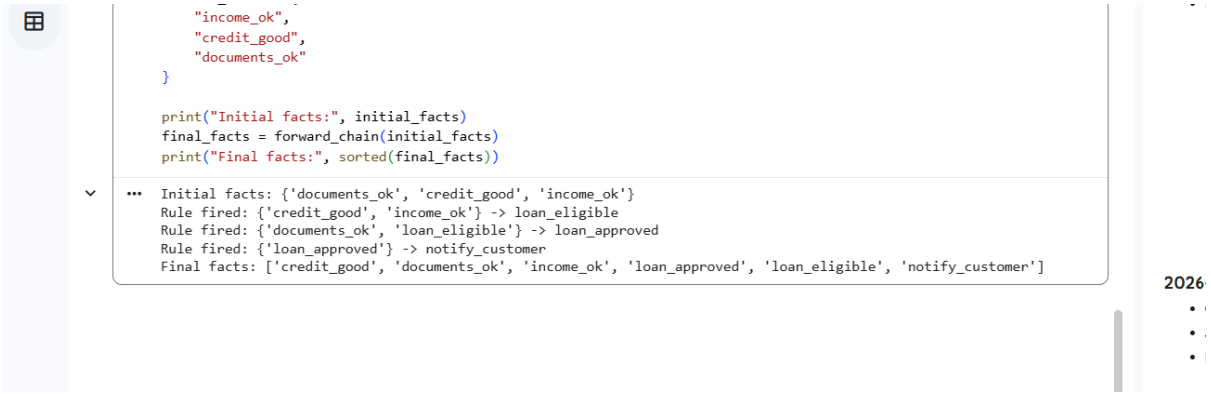
        if not fired:
            break

    return facts

initial_facts = {
    "income_ok",
    "credit_good",
    "documents_ok"
}

print("Initial facts:", initial_facts)
final_facts = forward_chain(initial_facts)
print("Final facts:", sorted(final_facts))
```

Output



```
    "income_ok",
    "credit_good",
    "documents_ok"
}

print("Initial facts:", initial_facts)
final_facts = forward_chain(initial_facts)
print("Final facts:", sorted(final_facts))
```

```
Initial facts: {'documents_ok', 'credit_good', 'income_ok'}
Rule fired: {'credit_good', 'income_ok'} -> loan_eligible
Rule fired: {'documents_ok', 'loan_eligible'} -> loan_approved
Rule fired: {'loan_approved'} -> notify_customer
Final facts: ['credit_good', 'documents_ok', 'income_ok', 'loan_approved', 'loan_eligible', 'notify_customer']
```

Experiment 5: Backward Chaining with Why/How Explanation

Aim

To implement goal-driven reasoning that asks for missing observations and explains why a question is asked and how a conclusion is reached.

Algorithm

1. Store rules as rule ID, conditions and conclusion.
2. Keep known true/false facts in a dictionary and define askable observations.
3. Implement recursive prove(goal): return known facts or try rules that conclude the goal.
4. When a required basic observation is unknown, ask the user for yes/no input.
5. If the user types why, explain the goal and rule that require the observation.
6. Record successful proof steps and display a How explanation for the final result.
7. Try different goals to demonstrate multiple conclusions and failure.

Program

```
rules = [
    (["yellow_leaves", "wet_soil"], "overwatering"),
    (["yellow_leaves", "dry_soil"], "underwatering"),
    (["brown_spots", "high_humidity"], "fungal_disease"),
    (["holes_in_leaves", "visible_insects"], "pest_attack")
]
```

```
askable = [
    "yellow_leaves", "wet_soil", "dry_soil",
    "brown_spots", "high_humidity",
    "holes_in_leaves", "visible_insects"
]

facts = {}

def prove(goal):
    if goal in facts:
        return facts[goal]

    relevant_rules = [r for r in rules if r[1] == goal]

    for conditions, conclusion in relevant_rules:
        print("Trying rule:", conditions, "->", conclusion)

        results = []
        for condition in conditions:
            results.append(prove(condition))

        if all(results):
            facts[goal] = True
            return True

    if goal in askable:
        while True:
            answer = input(
                f"Is {goal.replace('_', ' ')} present? (yes/no/why): "
            ).lower().strip()

            if answer == "why":
```

```
            print("WHY: This fact is needed to prove", goal)
            print("HOW: Relevant rules:", relevant_rules)
        elif answer in ["yes", "y"]:
            facts[goal] = True
            return True
        elif answer in ["no", "n"]:
            facts[goal] = False
            return False
        else:
            print("Enter yes, no, or why.")

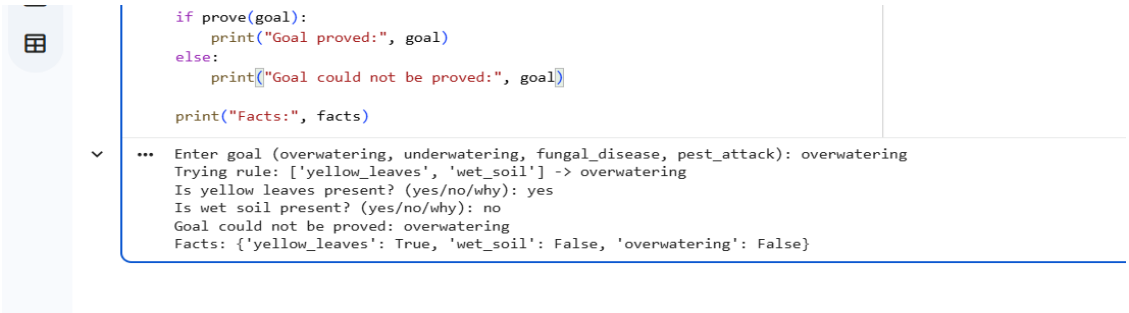
    facts[goal] = False
    return False

goal = input(
    "Enter goal (overwatering, underwatering, fungal_disease, pest_attack): "
).strip()

if prove(goal):
    print("Goal proved:", goal)
else:
    print("Goal could not be proved:", goal)

print("Facts:", facts)
```

Output



```
if prove(goal):
    print("Goal proved:", goal)
else:
    print("Goal could not be proved:", goal)

print("Facts:", facts)
```

```
Enter goal (overwatering, underwatering, fungal_disease, pest_attack): overwatering
Trying rule: ['yellow_leaves', 'wet_soil'] -> overwatering
Is yellow leaves present? (yes/no/why): yes
Is wet soil present? (yes/no/why): no
Goal could not be proved: overwatering
Facts: {'yellow_leaves': True, 'wet_soil': False, 'overwatering': False}
```

Experiment 6: Building and Reasoning over an Ontology

Aim

To create a small university ontology with classes, properties and individuals, then use a reasoner to infer class membership and check consistency.

Algorithm

1. Import owlready2 and create an ontology.
2. Define Person, Student, Faculty, Course, Department, Undergraduate and other classes.
3. Define object properties such as enrolledIn, teaches and belongsTo, plus data properties.
4. Create individuals and assign their names, courses and departments.
5. Define Student and Faculty as disjoint classes.
6. Run the HermiT reasoner to infer class membership and check consistency.
7. Print selected ontology facts and demonstrate an inconsistency if the reasoner supports it.

Program

```
from owlready2 import *


onto = get_ontology("http://example.org/university.owl")


with onto:
    class Person(Thing):
        pass


    class Student(Person):
```

```
    pass


class Faculty(Person):
    pass


class Course(Thing):
    pass


class enrolledIn(ObjectProperty):
    domain = [Student]
    range = [Course]


class hasName(DataProperty):
    domain = [Person]
    range = [str]


class Undergraduate(Student):
    pass


class Postgraduate(Student):
    pass


python_course = Course("Python_Course")
database_course = Course("Database_Course")


asha = Undergraduate("Asha")
ravi = Undergraduate("Ravi")
meena = Postgraduate("Meena")


asha.hasName = ["Asha"]
ravi.hasName = ["Ravi"]
meena.hasName = ["Meena"]
```

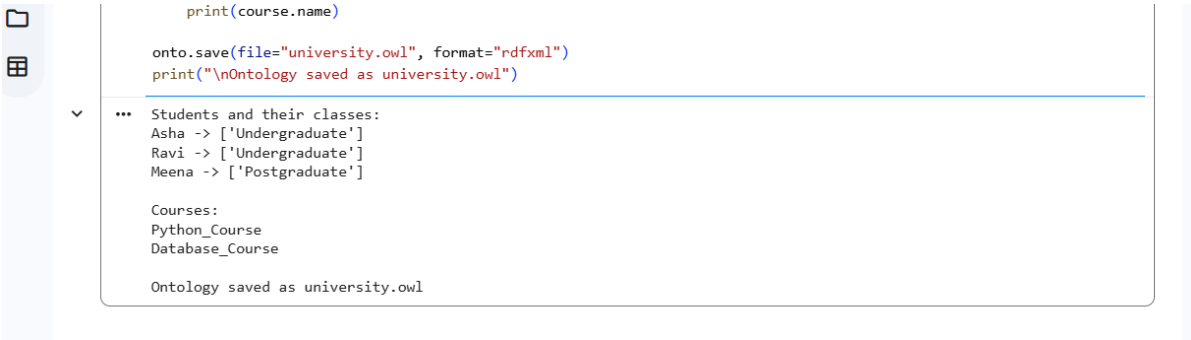
```
    asha.enrolledIn = [python_course]
    ravi.enrolledIn = [database_course]
    meena.enrolledIn = [python_course, database_course]

print("Students and their classes:")
for student in [asha, ravi, meena]:
    print(student.name, "->", [c.name for c in student.is_a])

print("\nCourses:")
for course in onto.Course.instances():
    print(course.name)

onto.save(file="university.owl", format="rdfxml")
print("\nOntology saved as university.owl")
```

Output



Experiment 7: Building and Querying a Knowledge Graph

Aim

To construct an RDF knowledge graph from structured records, merge a duplicate person, query triples and visualize graph connectivity.

Algorithm

1. Import pandas, rdflib, networkx and matplotlib.
2. Create people and work records as sample tables.
3. Convert each record into RDF triples for people, projects, departments and cities.
4. Compare similar names in the same department and merge a duplicate entity.
5. Run SPARQL queries to find project members and people in a department.
6. Convert RDF triples to a NetworkX graph, calculate degrees and draw the graph.
7. Save the knowledge graph in Turtle format.

Program

```
!pip -q install rdflib networkx matplotlib

from rdflib import Graph, Namespace, RDF, Literal

import networkx as nx

import matplotlib.pyplot as plt



EX = Namespace("http://example.org/")

g = Graph()

g.bind("ex", EX)



people = [

    ("Asha", "AI", "Chennai", "Project_Alpha"),

    ("Ravi", "CSE", "Hyderabad", "Project_Beta"),

    ("Meena", "AI", "Chennai", "Project_Alpha"),

    ("Arun", "ECE", "Bengaluru", "Project_Gamma")

]



for name, dept, city, project in people:

    person = EX[name]

    g.add((person, RDF.type, EX.Person))

    g.add((person, EX.department, EX[dept]))

    g.add((person, EX.city, Literal(city)))

    g.add((person, EX.worksOn, EX[project]))

    g.add((person, EX.name, Literal(name)))



print("Number of triples:", len(g))
```

```
query = """
PREFIX ex: <http://example.org/>
SELECT ?name WHERE {
    ?person ex:name ?name ;
            ex:department ex:AI .
}
"""

print("\nPeople in AI department:")
for row in g.query(query):
    print(row[0])

print("\nPeople working on Project Alpha:")
for person in g.subjects(EX.worksOn, EX.Project_Alpha):
    print(g.value(person, EX.name))

G = nx.Graph()

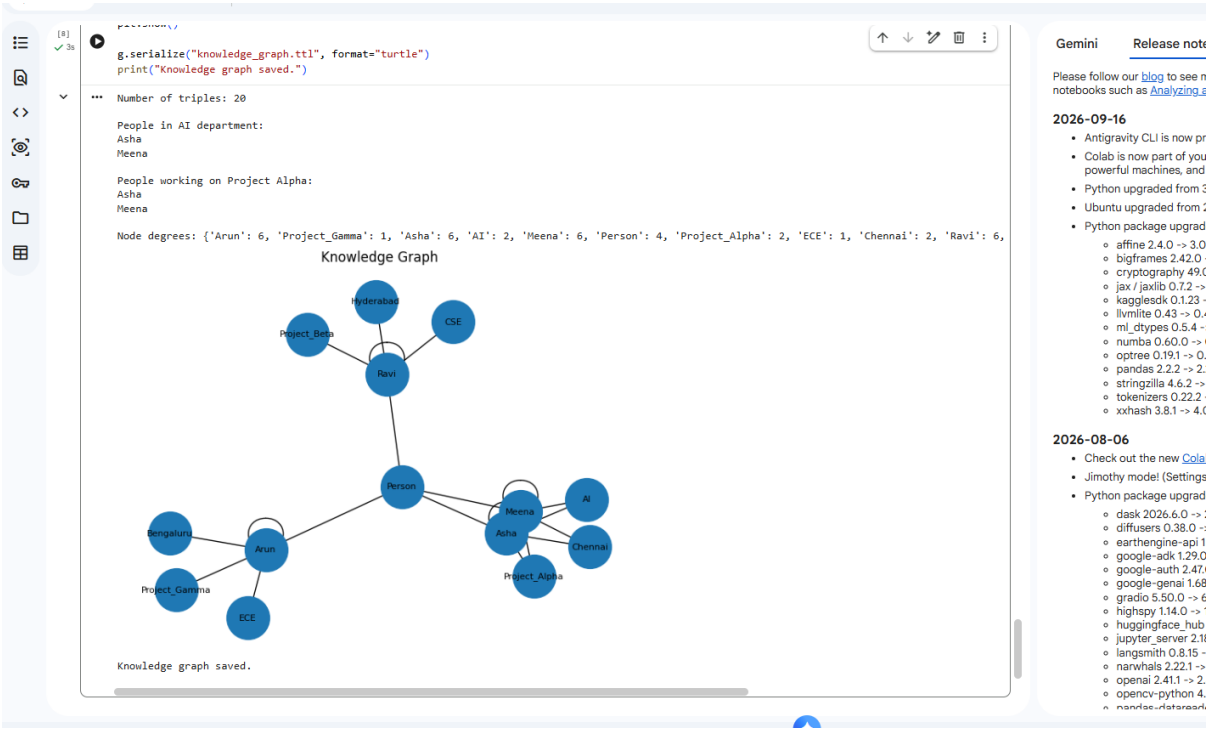
for subject, predicate, obj in g:
    G.add_edge(str(subject).split("/")[-1],
               str(obj).split("/")[-1],
               relation=str(predicate).split("/")[-1])

print("\nNode degrees:", dict(G.degree()))

nx.draw(G, with_labels=True, node_size=1500, font_size=8)
plt.title("Knowledge Graph")
plt.show()

g.serialize("knowledge_graph.ttl", format="turtle")
print("Knowledge graph saved.")
```

Output



Experiment 8: Hypothesis Testing, ANOVA and Chi-Square

Aim

To apply one-sample, independent and paired t-tests, one-way ANOVA and chi-square tests, and report p-values and a confidence interval.

Algorithm

1. Create/load exam scores for at least 60 students across methods A, B and C.
2. Set significance level alpha = 0.05 and state null and alternative hypotheses.
3. Perform one-sample, independent-samples and paired t-tests.
4. Perform one-way ANOVA across methods and chi-square independence test for gender and result.
5. Calculate the 95% confidence interval for score_after.
6. Print statistics, p-values, decisions and conclusions; draw a box plot.

Program

!pip -q install numpy pandas scipy matplotlib

import numpy as np

import pandas as pd

```
from scipy import stats
import matplotlib.pyplot as plt

np.random.seed(42)

n = 60
method = np.repeat(["A", "B", "C"], 20)
before = np.random.normal(55, 10, n)
improvement = np.concatenate([
    np.random.normal(8, 5, 20),
    np.random.normal(12, 5, 20),
    np.random.normal(16, 5, 20)
])

after = np.clip(before + improvement, 0, 100)
gender = np.random.choice(["F", "M"], n)
result = np.where(after >= 50, "Pass", "Fail")

df = pd.DataFrame({
    "method": method,
    "score_before": before,
    "score_after": after,
    "gender": gender,
    "result": result
})

alpha = 0.05

def report(name, test):
    print("\n", name)
    print("Statistic:", round(test.statistic, 4))
    print("P-value:", round(test.pvalue, 4))
```

```
    if test.pvalue < alpha:
        print("Decision: Reject null hypothesis")
    else:
        print("Decision: Fail to reject null hypothesis")


report("One-sample t-test", stats.ttest_1samp(after, popmean=60))


report("Independent t-test: A vs B",
       stats.ttest_ind(
           df[df.method == "A"]["score_after"],
           df[df.method == "B"]["score_after"],
           equal_var=False
       ))


report("Paired t-test",
       stats.ttest_rel(before, after))


groups = [
    df[df.method == m]["score_after"]
    for m in ["A", "B", "C"]
]
report("One-way ANOVA", stats.f_oneway(*groups))


table = pd.crosstab(df["gender"], df["result"])
chi2, p, dof, expected = stats.chi2_contingency(table)


print("\nChi-square statistic:", round(chi2, 4))
print("Chi-square p-value:", round(p, 4))
print(table)


mean = np.mean(after)
sem = stats.sem(after)
```

```
ci = stats.t.interval(0.95, df=n-1, loc=mean, scale=sem)

print("\n95% confidence interval:", ci)


df.boxplot(column="score_after", by="method")

plt.title("Scores After Teaching by Method")

plt.suptitle("")

plt.show()
```

Output

```
plt.show()
```

```
 One-sample t-test
Statistic: 3.7365
P-value: 0.0004
Decision: Reject null hypothesis

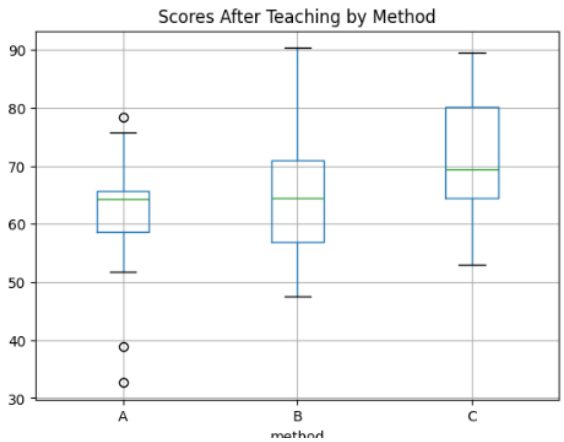
 Independent t-test: A vs B
Statistic: -0.8891
P-value: 0.3795
Decision: Fail to reject null hypothesis

 Paired t-test
Statistic: -15.8883
P-value: 0.0
Decision: Reject null hypothesis

 One-way ANOVA
Statistic: 4.439
P-value: 0.0162
Decision: Reject null hypothesis

Chi-square statistic: 0.0
Chi-square p-value: 1.0
result  Fail  Pass
gender
F          2    27
M          2    29

95% confidence interval: (np.float64(62.524441900002856), np.float64(68.34563016817252))
```



Experiment 9: Acquiring Rules from Data with Decision Trees

Aim

To learn IF–THEN rules from data using decision trees, evaluate them against a majority-class baseline and simple expert rules, and inspect overfitting.

Algorithm

1. Load the built-in Breast Cancer dataset.

2. Split data into training and testing sets with 70:30 stratified split and a fixed seed.

3. Calculate majority-class baseline accuracy.

4. Train decision trees with depths 2, 3, 4 and 6.

5. Record training, test and five-fold cross-validation accuracy for each depth.

6. Choose the best depth using cross-validation, print the learned rules and calculate evaluation metrics.

7. Plot accuracy against tree depth and discuss overfitting and limitations.

Program

```
!pip -q install scikit-learn pandas matplotlib

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt


from sklearn.datasets import load_breast_cancer

from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.tree import DecisionTreeClassifier, export_text

from sklearn.metrics import accuracy_score, confusion_matrix


data = load_breast_cancer()

X, y = data.data, data.target


X_train, X_test, y_train, y_test = train_test_split(
```

```
    X, y, test_size=0.30, random_state=42, stratify=y
)

depths = [2, 3, 4, 6]
results = []

for depth in depths:
    model = DecisionTreeClassifier(max_depth=depth, random_state=42)
    model.fit(X_train, y_train)

    train_acc = accuracy_score(y_train, model.predict(X_train))
    test_acc = accuracy_score(y_test, model.predict(X_test))
    cv_acc = cross_val_score(model, X_train, y_train, cv=5).mean()

    results.append([depth, train_acc, test_acc, cv_acc])

df = pd.DataFrame(
    results,
    columns=["Depth", "Train Accuracy", "Test Accuracy", "CV Accuracy"]
)

print("Accuracy comparison:")
print(df.round(3))

best_depth = int(df.loc[df["CV Accuracy"].idxmax(), "Depth"])
best = DecisionTreeClassifier(max_depth=best_depth, random_state=42)
best.fit(X_train, y_train)

pred = best.predict(X_test)

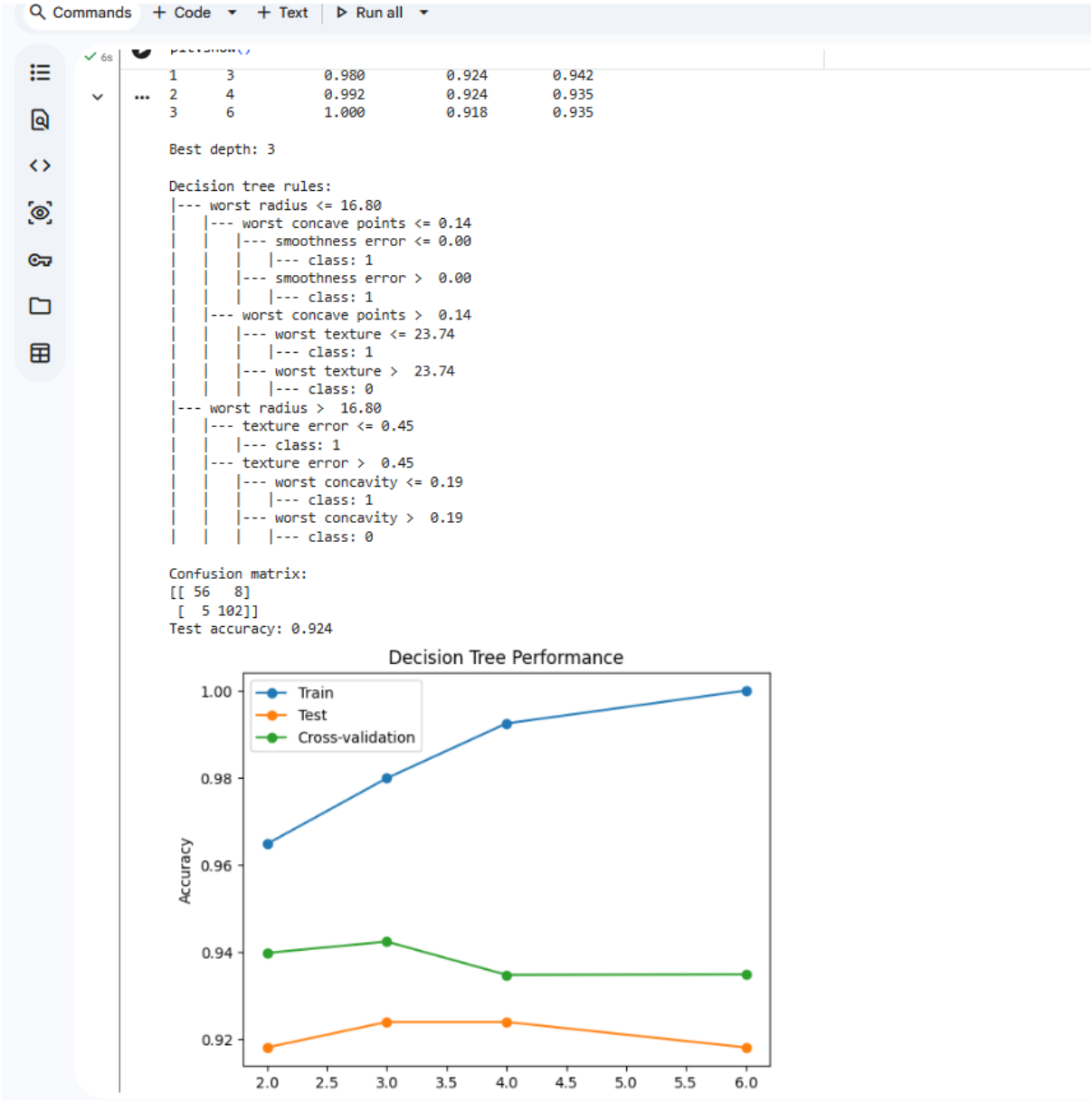
print("\nBest depth:", best_depth)
print("\nDecision tree rules:")
```

```
print(export_text(best, feature_names=list(data.feature_names)))


print("Confusion matrix:")
print(confusion_matrix(y_test, pred))
print("Test accuracy:", round(accuracy_score(y_test, pred), 3))


plt.plot(df["Depth"], df["Train Accuracy"], marker="o", label="Train")
plt.plot(df["Depth"], df["Test Accuracy"], marker="o", label="Test")
plt.plot(df["Depth"], df["CV Accuracy"], marker="o", label="Cross-validation")
plt.xlabel("Tree Depth")
plt.ylabel("Accuracy")
plt.title("Decision Tree Performance")
plt.legend()
plt.show()
```

Output



Experiment 10: Hybrid Decision-Support System (Knowledge Graph, Rules and ML)

Aim

To combine a small knowledge graph, expert rules and a machine-learning classifier to make an explainable crop recommendation.

Algorithm

1. Create a synthetic crop dataset with soil moisture, temperature and soil pH for at least 100 records.

2. Train and evaluate a decision-tree classifier using stratified cross-validation.

3. Create a knowledge graph containing crop facts and environmental relationships.

4. Implement expert rules with positive and negative evidence and certainty scores.

5. Combine the rule-based recommendation with the ML model prediction and confidence.

6. Print the facts, rules fired, model confidence and final recommendation for sample cases.

7. Evaluate the classifier and state that synthetic data is only for demonstration.

Program

```
!pip -q install numpy pandas networkx scikit-learn

import numpy as np

import pandas as pd

import networkx as nx


from sklearn.model_selection import train_test_split, cross_val_score

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, f1_score


rng = np.random.default_rng(7)

n = 150


moisture = rng.uniform(10, 90, n)

temperature = rng.uniform(15, 40, n)

ph = rng.uniform(4.5, 8.5, n)


def label_crop(m, t, p):

    if m >= 60 and 20 <= t <= 32 and 5.5 <= p <= 7.5:

        return "rice"

    elif m < 40 and 25 <= t <= 40 and 5 <= p <= 8:

        return "millet"

    else:

        return "maize"


y = np.array([

    label_crop(m, t, p)

    for m, t, p in zip(moisture, temperature, ph)
```

```
])

X = pd.DataFrame({
    "moisture": moisture,
    "temperature": temperature,
    "ph": ph
})

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y
)

model = DecisionTreeClassifier(max_depth=4, random_state=42)
model.fit(X_train, y_train)

pred = model.predict(X_test)

print("Test accuracy:", round(accuracy_score(y_test, pred), 3))
print("Macro F1-score:", round(f1_score(y_test, pred, average="macro"), 3))
print("Cross-validation accuracy:",
      round(cross_val_score(model, X_train, y_train, cv=5).mean(), 3))

# Create knowledge graph
KG = nx.DiGraph()
KG.add_edge("rice", "high_water_need")
KG.add_edge("millet", "low_water_need")
KG.add_edge("maize", "moderate_water_need")

print("\nKnowledge graph relationships:")
print(list(KG.edges()))

def recommend(m, t, p):
```

```
    sample = pd.DataFrame([{
        "moisture": m,
        "temperature": t,
        "ph": p
    }])

    prediction = model.predict(sample)[0]
    probabilities = model.predict_proba(sample)[0]
    confidence = max(probabilities)

    if m >= 60 and 20 <= t <= 32 and 5.5 <= p <= 7.5:
        recommendation = "rice"
        rule = "High moisture, warm temperature and suitable pH"
    elif m < 40 and t >= 25 and 5 <= p <= 8:
        recommendation = "millet"
        rule = "Low moisture and warm temperature"
    else:
        recommendation = prediction
        rule = "Machine-learning prediction"

    print("\nInput:", m, t, p)
    print("Rule applied:", rule)
    print("ML prediction:", prediction)
    print("ML confidence:", round(float(confidence), 3))
    print("Final crop recommendation:", recommendation)

recommend(75, 28, 6.5)
recommend(25, 33, 6.8)
recommend(48, 24, 6.2)

print("\nNote: This is a demonstration using synthetic data.")
```

Output

```
recommend(25, 33, 6.8)
recommend(48, 24, 6.2)

print("\nNote: This is a demonstration using synthetic data.")
```

```
Test accuracy: 0.868
Macro F1-score: 0.733
Cross-validation accuracy: 0.857

Knowledge graph relationships:
[('rice', 'high_water_need'), ('millet', 'low_water_need'), ('maize', 'moderate_water_need')]

Input: 75 28 6.5
Rule applied: High moisture, warm temperature and suitable pH
ML prediction: maize
ML confidence: 1.0
Final crop recommendation: rice

Input: 25 33 6.8
Rule applied: Low moisture and warm temperature
ML prediction: millet
ML confidence: 1.0
Final crop recommendation: millet

Input: 48 24 6.2
Rule applied: Machine-learning prediction
ML prediction: maize
ML confidence: 1.0
Final crop recommendation: maize

Note: This is a demonstration using synthetic data.
```